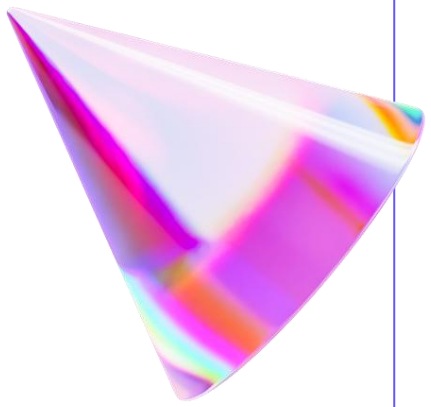
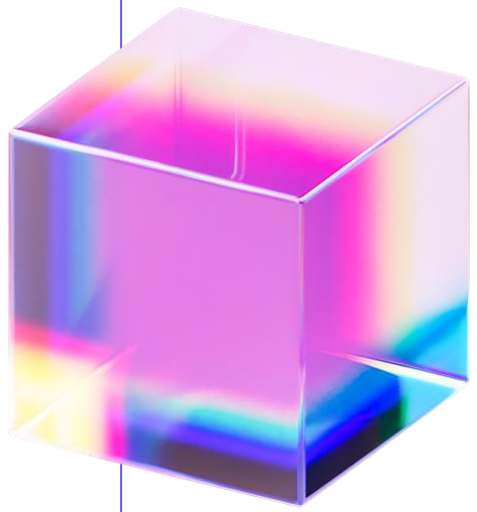
PYTHON PROJECT

BEHAVIOUR-AWARE PORTFOLIOS





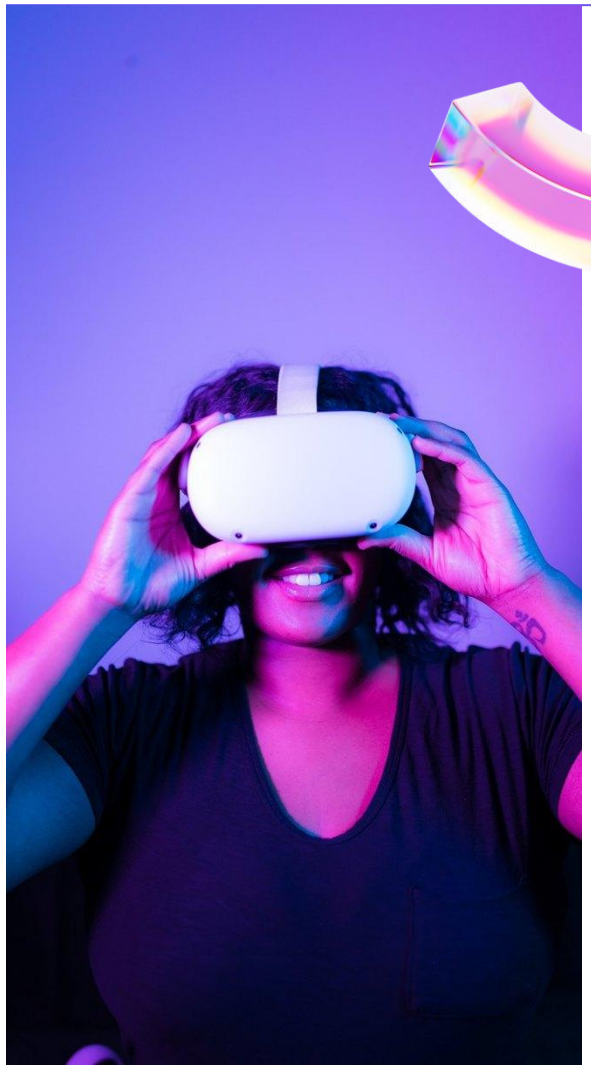


ABHAY S
246213044

SANKARSHANA MULUGUNDAM
246213080

TEN ANSWERS AND AN AMOUNT GO IN. A RUPEE SPLIT AND A MEASURE OF RISK COME OUT.

THE PROBLEM



WHAT THE PROGRAM READS

PRICES AND RETURNS
Daily, Jan 2016 to 13 Oct 2026 (Parquet)

VALUATION INPUTS
valuation.xlsx, cell by cell (openpyxl)

SETTINGS
config.yaml: risk-free rate, simulation sizes, SIP

TEXT
2,428 scored headlines; PDFs (pypdf)

WHAT IT GIVES BACK

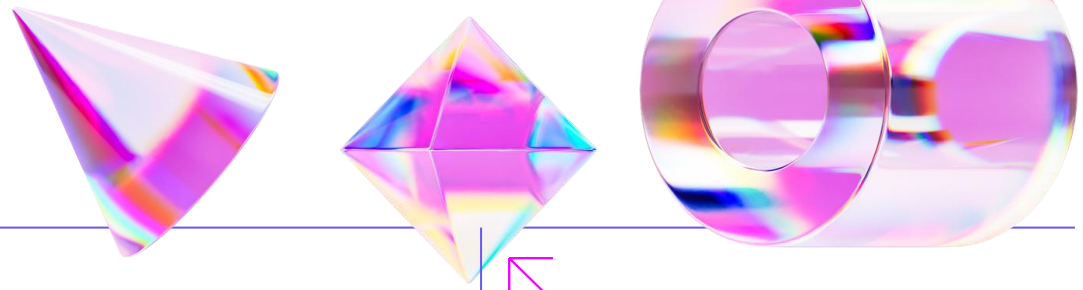
A = 4

BALANCED INVESTOR, ₹1,00,000

HUL	**₹13,382**
Sun Pharma	**₹8,394**
ICICI Bank	**₹21,833**
T-bills	**₹56,392**

One-year 95% VaR: ₹5,582

COMPUTATIONAL THINKING



DECOMPOSITION

Split one question into smaller ones: load and validate data, measure risk, value each stock, score the investor, optimise, simulate, report.

IN THE CODE

14 sections, one job each

PATTERN RECOGNITION

Volatility, correlation and beta in returns. Every valuation model has the same shape: cash flows plus a terminal value.

IN THE CODE

A shared base class

ABSTRACTION

Keep only what the decision needs. An Investor is a name, A, an amount and a SIP. A Security is a beta, a return and a volatility.

IN THE CODE

Three small classes

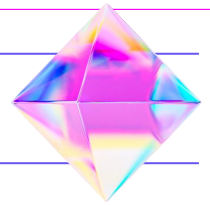
ALGORITHM DESIGN

Recursive bisection for implied growth. Merge sort and binary search for outcomes. Cholesky for correlated simulation.

IN THE CODE

Each checked another way

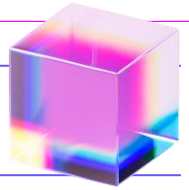
CONCEPT MAP



EVERY COURSE CONCEPT, MAPPED TO THE FILE. EACH ONE DOES REAL WORK IN THE PROJECT.

Concept	Section	What it does in the project
Variables, numbers, strings	1	Settings; Indian-style rupee formatting
Files and exceptions	2	JSON, YAML, CSV, Parquet, PDF; a clear DataError
Functions, Pandas	3	Returns, Sharpe, beta, Treynor, Jensen's alpha
Decisions, loops, dictionaries	4, 10	Questionnaire scoring; trading-rule signals
Classes	5	Security, Investor, Portfolio
Inheritance, recursion	6	ValuationModel and two subclasses; recursive PV, bisection
Sorting and searching	7	Merge sort, binary search, grid search
SciPy	8	Maximum Sharpe weights, efficient frontier
NumPy	9, 10	Cholesky Monte Carlo, block bootstrap, EWMA, SIP
Sets, Transformers, pypdf, yfinance	2, 11	Hedge words, FinBERT, PDF text, price download
Matplotlib, Seaborn	12	Frontier, fan chart, correlation heatmap
Streamlit	14	A small interactive demo in the same file

INHERITANCE



ONE BASE MODEL, TWO SUBCLASSES. A SUBCLASS ONLY SAYS WHAT ITS CASH FLOWS ARE.

```python
class DividendDiscountModel(ValuationModel):

    def __init__(self, name, price, r, g2, n, d0: float, g1: float):
        super().__init__(name, price, r, g2, n)
        self.d0, self.g1 = d0, g1

    def cash_flows(self) -> list[float]:
        flows, dividend = [], self.d0
        for year in range(1, self.HORIZON + 1):
            dividend *= 1 + (self.g1 if year <= self.n else self.g2)
            flows.append(dividend)
        return flows

    def terminal_value(self) -> float:
        return self.cash_flows()[-1] * (1 + self.g2) / (self.r - self.g2)
```

INHERITED, NOT WRITTEN HERE

present_value and implied_growth both come from the base class. Only the cash flows and the terminal value differ.

ValuationModel (abstract)
present_value, implied_growth

DividendDiscountModel
HUL: pays out most of its profit

ResidualIncomeModel
Sun Pharma, ICICI Bank

BUSINESS ENTITIES

Security
dataclass: ticker, name, beta, return, volatility

Investor
name, A, amount, SIP; from_answers classmethod

Portfolio
checks weights sum to 1; complete, allocation, ranked

RECURSION

TWO RECURSIVE FUNCTIONS, EACH CHECKED AGAINST ALGEBRA AND THE EXCEL WORKBOOK.

```python
def present_value(flows: list[float], r: float, t: int = 1) -> float:
    if not flows:           # base case: nothing left to discount
        return 0.0
    return flows[0] / (1 + r) ** t + ValuationModel.present_value(flows[1:], r, t + 1)

def implied_growth(self, low, high, tol):
    gap_low = self.single_stage_value(low) - self.price
    middle = (low + high) / 2
    if high - low < tol:    # base case: the range is small enough
        return middle
    if gap_low * (self.single_stage_value(middle) - self.price) <= 0:
        return self.implied_growth(low, middle, tol)
    return self.implied_growth(middle, high, tol)
```

BASE CASE

An empty list of cash flows is worth 0. For bisection, a range narrower than 1e-10 is the answer.

SMALLER PROBLEM

One fewer cash flow, one year later. Bisection halves its range on every call.

PROOF

A test asserts recursive value = closed form = Excel, and bisection = algebra.

Stock	Class	Value	Implied growth
HUL	Dividend	₹1,079.12	7.41%
Sun Pharma	Residual income	₹747.63	8.63%
ICICI Bank	Residual income	₹574.65	13.89%

6 implied_growth shortened to fit: its default range and its no-solution check are left out.

EXCEPTIONS

FILES AND DATA STRUCTURES. BAD DATA STOPS THE PROGRAM, WITH A CLEAR MESSAGE.

```python
def read_returns(path: Path, needed: list[str]) -> pd.DataFrame:
    try:
        table = pd.read_parquet(path)
    except FileNotFoundError as err:
        raise DataError(f"{path} not found; run the pipeline first") from err
    missing = [c for c in needed if c not in table.columns]
    if missing:
        raise DataError(f"{path} lacks columns {missing}")
    gaps = int(table[needed].isna().sum().sum())
    if gaps:
        raise DataError(f"{path} has {gaps} missing values in {needed}")
    return table[needed]
```

DICTIONARY

QUESTIONS maps each question to its kind, text and five options.

LIST

The answers for capacity and for willingness, kept separately.

SET

HEDGE_WORDS: fast lookup of cautious words like may, might, could.

REAL OUTPUT FROM THE TOUR

A missing file is caught, not ignored:

```
...does_not_exist.parquet not found; run the pipeline first
```

Bad inputs are refused:

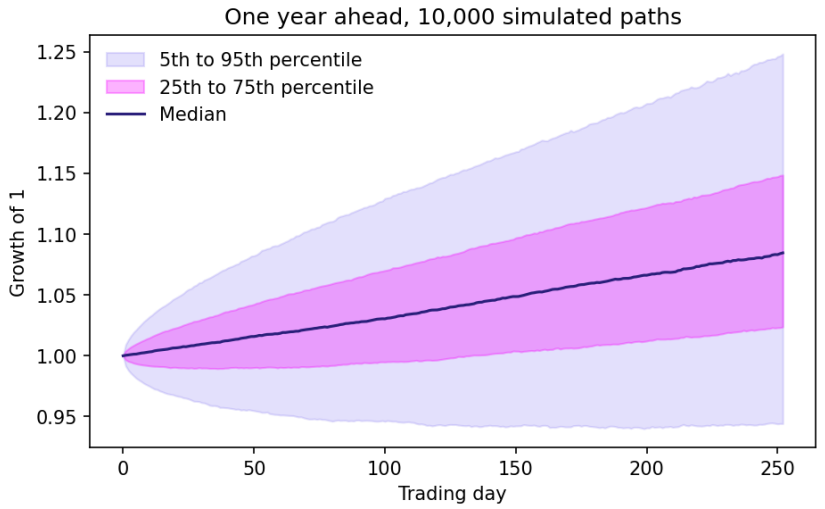
```
Bad input: the discount rate 5.00% must exceed terminal growth 6.00%
```

SIMULATION

NUMPY: 10,000 SIMULATED YEARS, WITHOUT A PYTHON LOOP OVER THE PATHS.

```python
def simulate_normal(mu, cov, n_sims, horizon, rng):
    chol = np.linalg.cholesky(cov)
    return rng.standard_normal((n_sims, horizon, len(mu))) @ chol.T + mu

def wealth_paths(risky_returns, y, rf_period):
    growth = np.cumprod(1 + y * risky_returns + (1 - y) * rf_period, axis=1)
    return np.hstack([np.ones((risky_returns.shape[0], 1)), growth])
```



₹1,08,463

MEDIAN AFTER ONE YEAR, ₹1 LAKH, A = 4

₹5,582

95% VaR, NORMAL RETURNS

₹5,465

95% VaR, BLOCK BOOTSTRAP

CHOLESKY

Split the covariance matrix so that cov = L L^T. Independent draws multiplied by L have the correlations we measured.

Half of all paths end between ₹1,02,377 and ₹1,14,860.

Type hints trimmed to fit the slide.

OPTIMISATION

SCIPY FINDS THE MAXIMUM-SHARPE WEIGHTS. A 1% GRID SEARCH FINDS THE SAME PORTFOLIO.

```python
def max_sharpe_weights(mu, cov, rf: float, bounds=WEIGHT_BOUNDS) -> np.ndarray:
    mu, cov = np.asarray(mu), np.asarray(cov)
    return _optimise(lambda w: -(w @ mu - rf) / np.sqrt(w @ cov @ w), len(mu), bounds)

def _optimise(objective, n: int, bounds, constraints=()) -> np.ndarray:
    all_constraints = [{"type": "eq", "fun": lambda w: w.sum() - 1.0}, *constraints]
    result = minimize(objective, np.full(n, 1.0 / n), method="SLSQP", bounds=[bounds] * n,
                      constraints=all_constraints, options={"ftol": 1e-12, "maxiter": 500})
```

SCIPY SLSQP

31 / 19 / 50

Sharpe 0.3436, within 10–60% limits

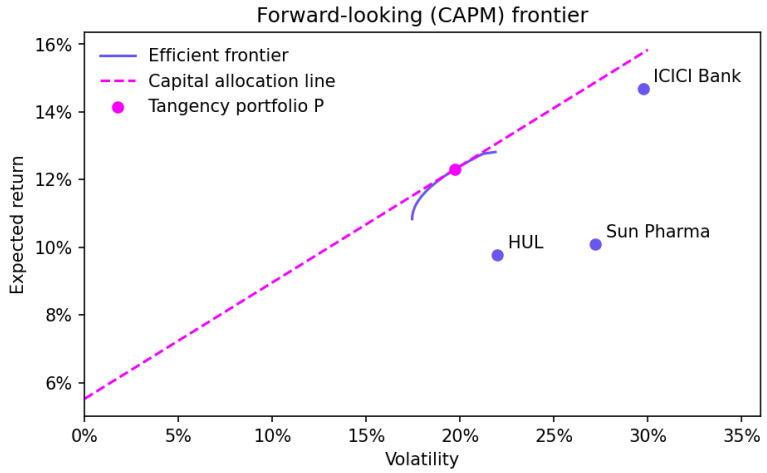
1% GRID SEARCH

31 / 19 / 50

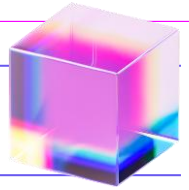
Same portfolio, by trying every mix

THEN SIZE IT FOR EACH INVESTOR

$y^* = (E[r_P] - r_f) / (A\sigma^2)$: 87% at A = 2, 44% at A = 4, 29% at A = 6. The rest goes to T-bills.



SORT AND SEARCH



MERGE SORT RANKS THE OUTCOMES. BINARY SEARCH FINDS HOW MANY END BELOW ₹1 LAKH.

```python
def merge_sort(items: list, key=lambda x: x) -> list:
    if len(items) <= 1:
        return list(items)
    middle = len(items) // 2
    left, right = merge_sort(items[:middle], key), merge_sort(items[middle:], key)
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    return merged + left[i:] + right[j:]
```

```python
def binary_search(ordered, value):
    low, high = 0, len(ordered)
    while low < high:
        middle = (low + high) // 2
        if ordered[middle] < value:
            low = middle + 1
        else:
            high = middle
    return low
```

Each step halves the list: about 14 steps. Scanning it could take 10,000.

WORST THREE

₹78,719, ₹79,877, ₹80,171

BEST THREE

₹1,46,320, ₹1,46,757, ₹1,47,604

BELOW ₹1 LAKH

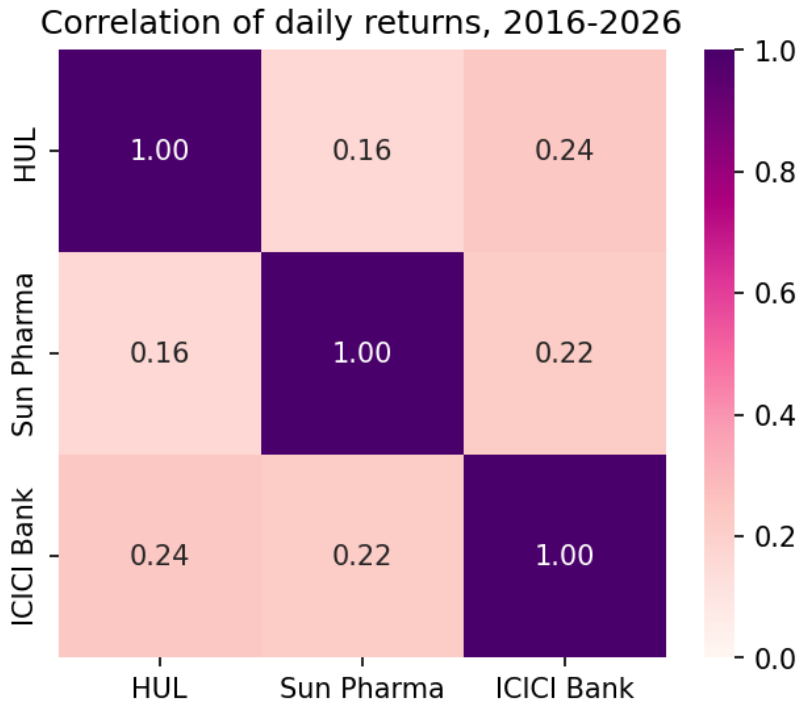
17.1% of paths

10 Merge sort checked against sorted() on 2,000 paths; sorted() ranks all 10,000. A = 4, ₹1 lakh, one year. Type hints trimmed.

LIBRARIES

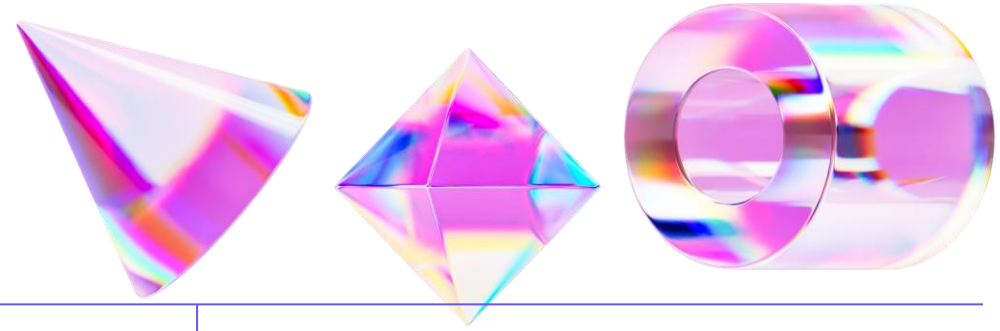
EACH LIBRARY HAS ONE JOB. HEAVY ONES LOAD ONLY INSIDE THE FUNCTION THAT NEEDS THEM.

Library	Job
Pandas	Load, align and summarise returns; metrics tables
Matplotlib	Efficient frontier and Monte Carlo fan chart
Seaborn	Correlation heatmap
yfinance	Fresh prices (--fetch)
Transformers (FinBERT)	Financial tone of a sentence (--finbert)
pypdf	Text from annual reports and transcripts (--pdf)
openpyxl	Excel valuation inputs, cell by cell
Streamlit	Interactive demo on one screen
requests	Optional live answers through OpenRouter



RESULTS

ONE PORTFOLIO, THREE INVESTORS. ONLY THE SHARE IN IT CHANGES.



AGGRESSIVE · A = 2

87%

IN THE THREE STOCKS

Expected return	**11.4%**
Volatility	**17.2%**
95% VaR on ₹1 lakh	**₹16,138**
Chance of loss	**28%**
15-year SIP median	**₹69.5 lakh**

BALANCED · A = 4

44%

IN THE THREE STOCKS

Expected return	**8.5%**
Volatility	**8.6%**
95% VaR on ₹1 lakh	**₹5,582**
Chance of loss	**17%**
15-year SIP median	**₹64.8 lakh**

CONSERVATIVE · A = 6

29%

IN THE THREE STOCKS

Expected return	**7.5%**
Volatility	**5.7%**
95% VaR on ₹1 lakh	**₹1,940**
Chance of loss	**10%**
15-year SIP median	**₹61.3 lakh**

12 ₹1 lakh for one year, then a ₹10,000 monthly SIP rising 10% a year for 15 years. Simulated; not investment advice.

TESTING

SAME ANSWERS

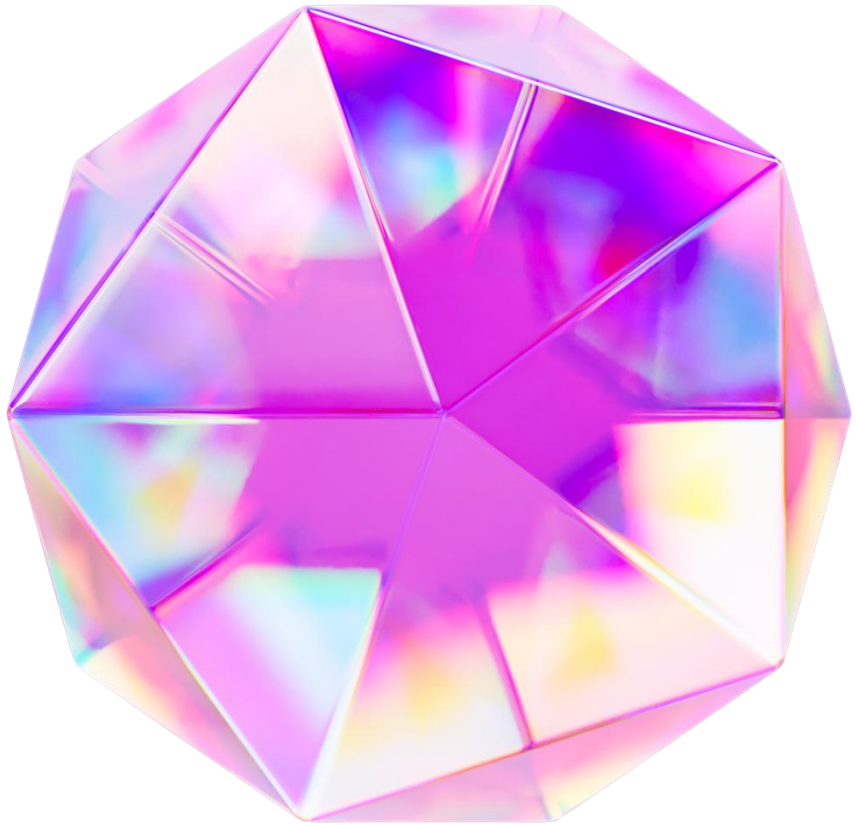
Valuations equal the Excel workbook. Metrics and weights equal the pipeline's files. Simulations reproduce mc.json and sip.json exactly.

SECOND METHODS

Recursion and bisection equal closed forms. Grid search finds SciPy's optimum. Our merge sort equals Python's sorted.

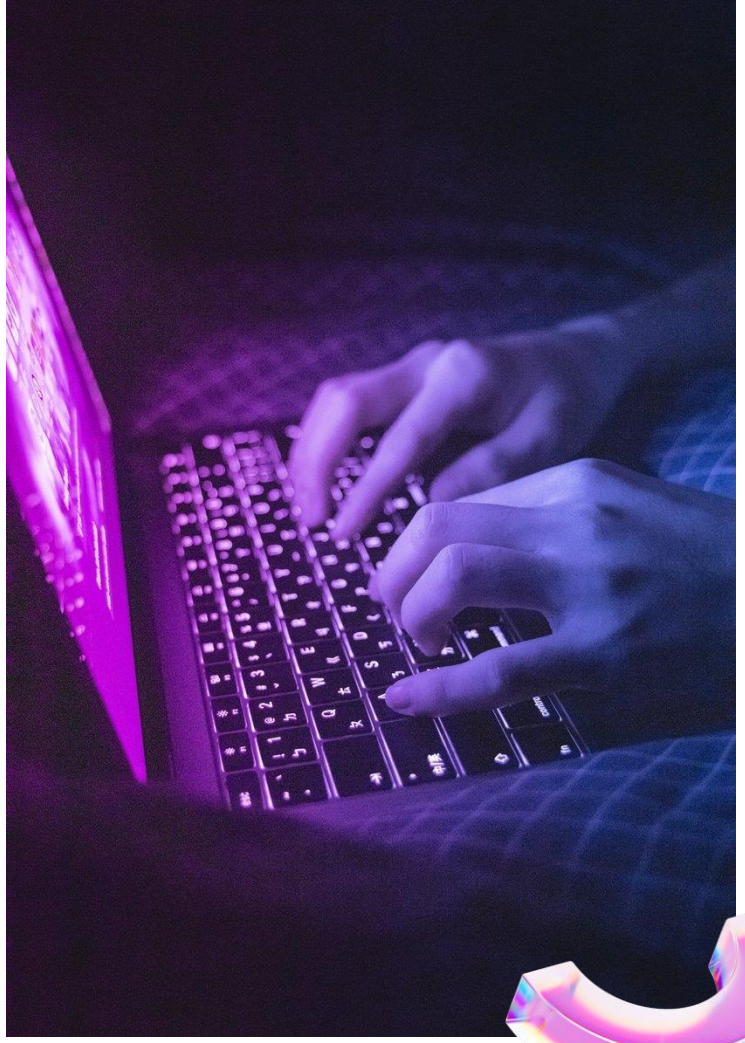
BAD INPUTS

A discount rate below growth, weights not summing to 1, a blank Excel cell, and missing or broken files all fail with a clear message.



```
python -m pytest tests/test_python_project.py
```

21 passed



`python python_project.py`

The full tour: each result beside the pipeline's, marked OK

`python python_project.py --charts out/`

Saves the Matplotlib and Seaborn charts

`streamlit run python_project.py`

Move A from 4 to 2: the share in P goes from 44% to 87%

`python -m pytest tests/test_python_project.py`

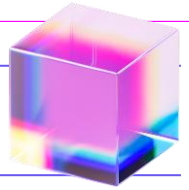
21 tests, in a few seconds

LIVE DEMO

IF A COMMAND FAILS

Show the saved output and charts in deliverables/assets/python, and keep going.

HONEST NOTES



WHAT CHANGED FROM THE APPROVED PROPOSAL, AND HOW WE USED AI, SO NOTHING IS HIDDEN.

Proposal said	The project does	Why
FINRA NCFS investor microdata	A 10-question questionnaire (capacity and willingness)	It sets A for one investor directly; we did not calibrate it on NCFS
Loughran–McDonald word lists	A small hedge-word set of the same kind	Keeps the file self-contained; it is not the full list
NSE total-return index	Nifty 50 price plus an estimated 1.22% a year	Estimated from the gap between Nifty BeES and the index
Anthropic SDK (optional)	A free model through OpenRouter (optional)	No cost for a live demo; every number still comes from our files

AI ASSISTANCE

Claude helped write and review this code. We checked it by running it, by testing it against Excel and the pipeline, and by reading it until we could explain each function; docs/python_walkthrough.md is our map. This is coursework, not investment advice.

THANK YOU

